Supervised learning

Linear Regression:

Linear Regression is a supervised machine learning algorithm that predicts a continuous target variable using one or more independent variables.

Types of Linear Regression

1. **Simple Linear Regression**:
   - Involves one independent variable and one dependent variable.
   - Equation:

     $y = mx + c$
     - y: Dependent variable
     - x: Independent variable
     - m: Slope (coefficient)
     - c: Intercept
2. **Multiple Linear Regression**:
   - Involves two or more independent variables.
   - Equation:

     $y = b_o + b_1x_1 + b_2x_2 + ... + b_nx_n$
     - b_o: Intercept
     - $b_1, b_2, ..., b_n$: Coefficients for each variable

       $x_1, x_2, ..., x_n$

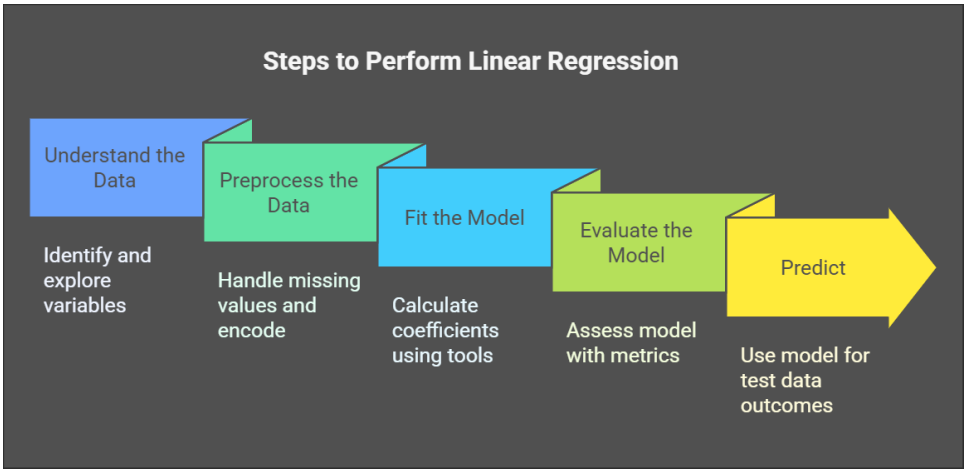
Assumptions of Linear Regression

1. **Linearity**: The relationship between independent and dependent variables must be linear.
2. **Independence**: Observations must be independent of each other.

Key Terms

- **Dependent Variable**: The outcome we want to predict (response variable).
- **Independent Variables**: The features used to predict the outcome (predictors).
- **Residuals**: The difference between actual and predicted values.yactual − ypredicted
- **Cost Function**: Measures prediction error, typically using Mean Squared Error (MSE):MSE = 1/n ∑(yi − ŷi)²

Steps to Perform Linear Regression



1. **Understand the Data**:
   - Identify independent and dependent variables.
   - Explore the data through descriptive statistics and visualization.
2. **Preprocess the Data**:
   - Handle missing values.
   - Encode categorical variables if present.
   - Scale or standardize features when needed.
3. **Fit the Model**:
   - Calculate coefficients using training data.
   - Tools: Python libraries like `scikit-learn` or `statsmodels`.
4. **Evaluate the Model**:
   - Use metrics such as:

    - **R-squared**: Proportion of variance explained by the model.
    - **Adjusted R-squared**: R-squared adjusted for the number of predictors.
    - **Mean Absolute Error (MAE)**.
    - **Root Mean Squared Error (RMSE)**.
5. **Predict**:
    - Use the model to predict outcomes for test data.

Advantages of Linear Regression

- Simple to understand and interpret.
- Efficient for small datasets.
- Provides insights into relationships between variables.

Limitations of Linear Regression

- Sensitive to outliers.
- Cannot model complex, non-linear relationships.
- Assumes strict linearity, which may not hold for real-world data.

Applications

1. Predicting sales, prices, and trends.
2. Risk analysis and financial forecasting.
3. Medical research (e.g., predicting patient outcomes).
4. Marketing analytics.

Least Squares:

- **Least Squares** is a mathematical optimization approach used to minimize the sum of the squares of residuals (errors).
- Commonly used for **fitting linear models** in machine learning, specifically in **Linear Regression**.
- The goal of **Least Squares** is to find the best-fit line (or hyperplane in multiple dimensions) that minimizes the discrepancy between the predicted

values and actual values (residuals).

Concepts

- **Residuals**: Difference between observed (y_i) and predicted ($\hat{y}_i$) values:
  $\varepsilon_i = y_i - \hat{y}_i$.
- **Sum of Squared Errors (SSE)**:
  $\text{SSE} = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$.

Model Representation:

- **Linear Regression Model**:
  $Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_n X_n + \varepsilon$.
  - Y: Dependent variable.
  - X: Independent variables.
  - β: Coefficients to be estimated.
  - ε: Error term.

Steps in Least Squares Method

1. **Define the Model**: Choose a linear model (e.g., $Y = \beta_0 + \beta_1 X$).
2. **Compute Residuals**: Calculate $\varepsilon_i = y_i - \hat{y}_i$.
3. **Minimize SSE**: Find coefficients $\beta_0, \beta_1, \ldots, \beta_n$ that minimize $\sum \varepsilon_i^2$.

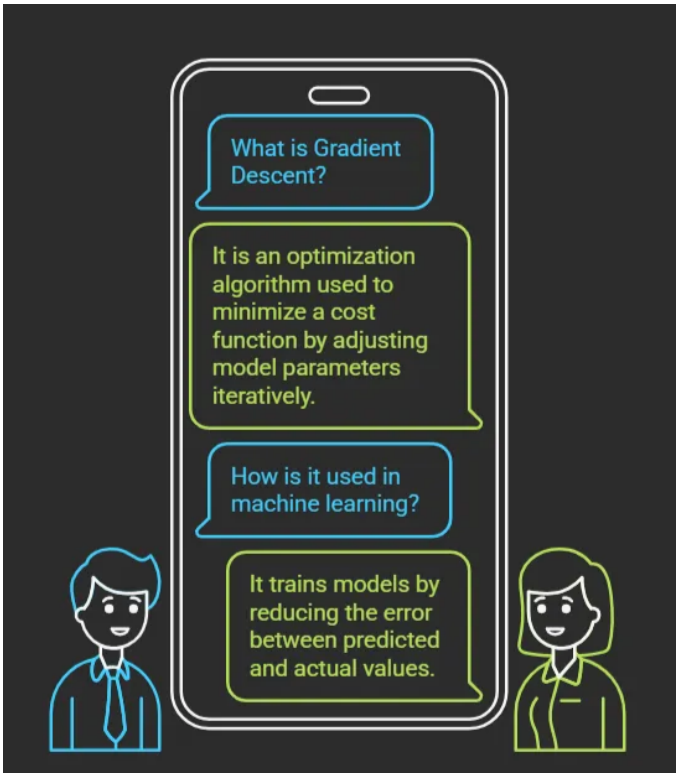
Solution

- **Simple Linear Regression**:
  $\beta_1 = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$,
  $\beta_0 = \bar{y} - \beta_1 \bar{x}$.
- **Multiple Linear Regression**:
  Use matrix form: $\boldsymbol{\beta} = (X^T X)^{-1} X^T Y$.

Applications:

- **Regression Analysis**: Predicting continuous outcomes (e.g., house prices, sales).
- **Curve Fitting**: Fitting a curve to data points.
- **Parameter Estimation**: Estimating model parameters in machine learning.

Gradient Descent:



How it Works

The algorithm updates the model parameters in the direction of the steepest descent of the cost function.

Parameter Update Rule

$$\theta_j = \theta_j - \alpha \cdot \frac{\partial J(\theta)}{\partial \theta_j}$$

Where:

- θ_j: Parameter being updated
- α: Learning rate (step size)
- $\frac{\partial J(\theta)}{\partial \theta_j}$: Partial derivative of the cost function $J(\theta)$ with respect to θ_j

Cost Function

The cost function measures the error between the predicted and actual values. For linear regression, the cost function is the **Mean Squared Error (MSE)**:

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^{m} (h_\theta(x_i) - y_i)^2$$

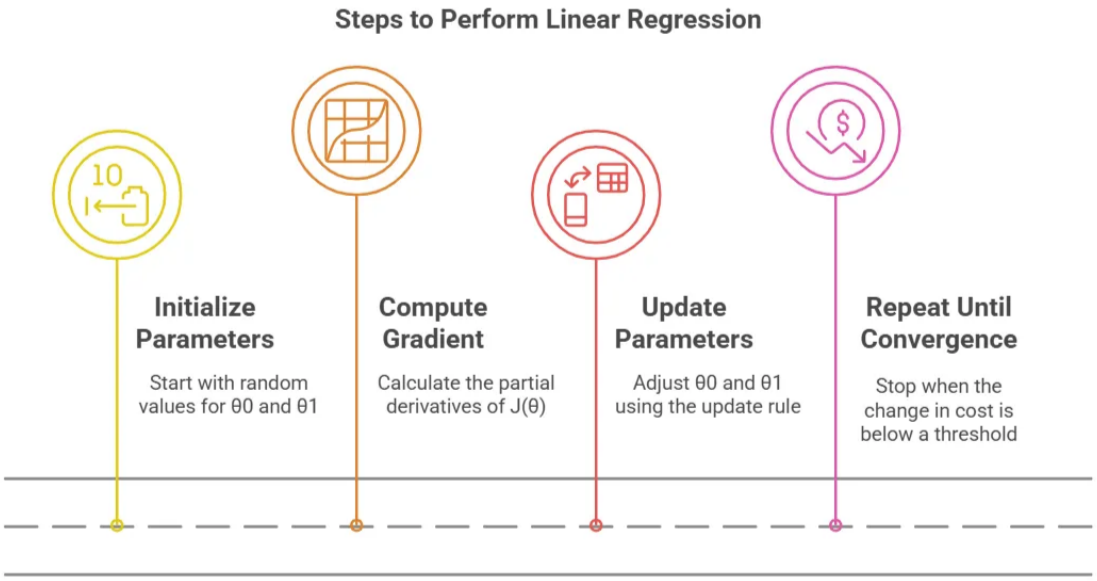
Where:

- m: Number of training examples
- $h_\theta(x_i)$: Predicted value
- y_i: Actual value

Gradient Descent for Linear Regression Example

Steps:

1. **Initialize Parameters**: Start with random values for θ0 and θ1.
2. **Compute Gradient**: Calculate the partial derivatives of J(θ).
3. **Update Parameters**: Adjust θ0 and θ1 using the update rule.
4. **Repeat Until Convergence**: Stop when the change in cost is below a threshold.



Applications of Gradient Descent

1. Training machine learning models (e.g., linear regression, neural networks).
2. Minimizing complex cost functions in optimization problems.

Linear Classification Models:

Linear classification models are algorithms used to separate data points into distinct classes based on a linear decision boundary. They are simple, efficient, and widely used for binary and multi-class classification tasks.

Key Concepts

1. **Decision Boundary**
   - A hyperplane (line in 2D, plane in 3D) that separates the data points into different classes.
   - For binary classification, the model predicts class labels .

     $y \in \{0,1\}$
2. **General Equation**

   y= w^T x + b

   Where:
   - x: Input features
   - w: Weights (determines the orientation of the boundary)
   - b: Bias (shifts the boundary)

Popular Linear Classification Models

1. **Logistic Regression**
   - Uses the **sigmoid function** to map predictions to probabilities:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

   - Suitable for binary and multi-class classification (via one-vs-rest or softmax).

2. **Linear Discriminant Analysis (LDA)**
    - Assumes normal distribution of features and equal class covariance.
    - Maximizes separation between classes using Bayes' theorem.
3. **Support Vector Machines (SVM)** (Linear Kernel)
    - Maximizes the **margin** (distance between the decision boundary and nearest data points).
    - Robust to outliers with the use of soft margins.
4. **Perceptron**
    - A basic linear classifier that updates weights based on misclassified points.
    - Works only for linearly separable data.

Assumptions

1. Linearly separable data (or approximately linear separation).
2. Independent and identically distributed (i.i.d.) data points.

Applications

- Spam email detection (binary classification).
- Credit card fraud detection.
- Sentiment analysis (e.g., positive/negative reviews).
- Image classification with linear features.

Discriminant Function and Perceptron Algorithm:

Discriminant Function

A **discriminant function** is a mathematical function used in classification tasks to determine the class of a given input. For linear classifiers, it defines a linear decision boundary that separates classes.

General Form of a Linear Discriminant Function

$g(x) = w^T x + b$

Where:

- x: Input feature vector
- w: Weight vector (defines the orientation of the decision boundary)
- b: Bias (defines the position of the decision boundary)
- g(x): Discriminant value

Classification Rule

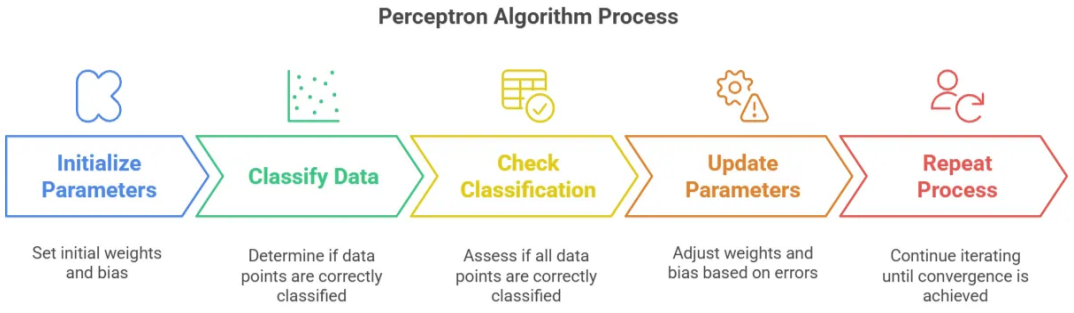
For a binary classification problem ($y \in \{-1, 1\}$):

- $g(x) > 0$: Class $+1$
- $g(x) \leq 0$: Class -1

The goal is to adjust w and b such that the decision boundary correctly separates the data points.

Perceptron Algorithm

The **Perceptron Algorithm** is an iterative learning algorithm used to find a linear discriminant function for linearly separable data. It updates the model parameters (w and b) based on classification errors.



Key Idea

Adjust the weights w and bias b such that misclassified points are moved closer to their correct side of the decision boundary.

Algorithm Steps

1. **Initialization**
   - Start with $w = 0$ and $b = 0$ (or small random values).
2. **Prediction**
   - Compute the predicted class using:
   $$\hat{y}_i = \text{sign}(w^T x_i + b)$$
3. **Weight and Bias Update Rule**
   For each misclassified point ($y_i \neq \hat{y}_i$):
   $$w = w + \eta y_i x_i$$
   $$b = b + \eta y_i$$
   Where:
   - y_i: True label ($+1$ or -1)
   - x_i: Input vector
   - η: Learning rate (controls the step size)
4. **Repeat**
   - Iterate over the dataset until all points are correctly classified or a maximum number of iterations is reached.

Convergence

The Perceptron Algorithm converges to a solution if the data is linearly separable. If not, it may fail to converge or oscillate indefinitely.

Limitations

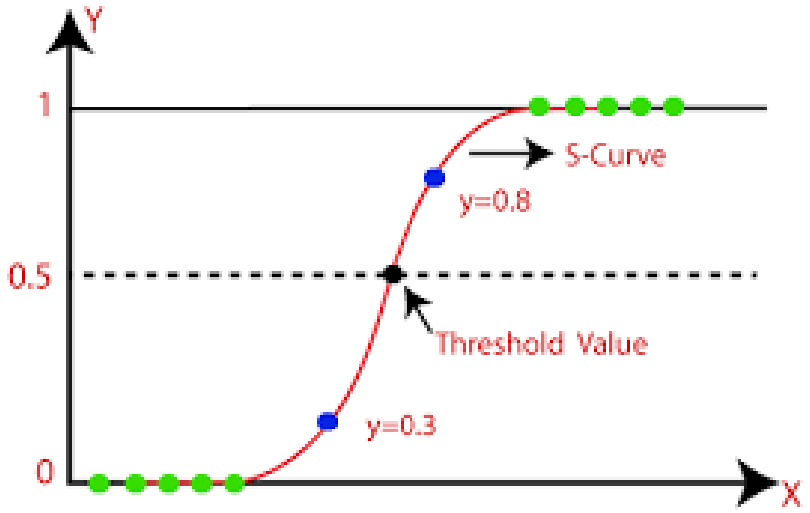
1. Does not converge for non-linearly separable data.
2. No probabilistic interpretation (unlike logistic regression).

Applications

1. Binary classification tasks (e.g., spam detection, sentiment analysis).
2. Initial foundations for more advanced algorithms like Support Vector Machines (SVMs).

Probabilistic Discriminative Model: Logistic Regression

Logistic Regression is a **probabilistic discriminative model** used for classification tasks. It predicts the probability of a data point belonging to a specific class and uses a logistic (sigmoid) function to map predicted values to probabilities. It is primarily used for **binary classification** but can be extended to multi-class problems.



Key Concepts

1. **Decision Boundary**
   Logistic Regression uses a linear decision boundary to separate classes. The decision is based on the probability threshold (default: 0.5).
2. **Hypothesis Function**
   The model predicts the probability that a given input x belongs to class $y = 1$:

$$h_\theta(x) = P(y = 1|x) = \sigma(w^T x + b)$$

   Where:

   - w: Weights
   - b: Bias
   - $\sigma(z)$: Sigmoid function

3. **Sigmoid Function**
   The sigmoid function maps any real value to the range [0, 1]:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

   - If $h_\theta(x) \geq 0.5$, predict $y = 1$.
   - Otherwise, predict $y = 0$.

Steps for Logistic Regression

1. **Data Preparation**
   - Prepare the dataset and normalize features for faster convergence.
2. **Model Hypothesis**
   - Define the sigmoid function for the output probabilities.
3. **Cost Function**
   - Define the log-loss function to measure model performance.
4. **Gradient Descent**
   - Update parameters iteratively using gradient descent to minimize the cost function.
5. **Prediction**
   - Classify the output based on the decision threshold (e.g., 0.5).

Advantages

1. Outputs probabilities, providing more information than binary class labels.
2. Works well when the relationship between features and the target is linear.

Limitations

1. Assumes a linear decision boundary, which may not work well for non-linear relationships.
2. Requires sufficient data to avoid overfitting.

Applications

1. Binary classification problems:
   - Email spam detection.
   - Disease diagnosis (e.g., predicting heart disease).
2. Multi-class classification using one-vs-rest or softmax.

Probabilistic Generative Model: Naive Bayes

Naive Bayes is a **probabilistic generative model** used for classification tasks. It is based on **Bayes' theorem** and assumes that the features are conditionally independent given the class label. Despite its simplicity, Naive Bayes often performs well in practical scenarios, especially for text classification and spam filtering.

Bayes' Theorem

Bayes' theorem forms the foundation of the Naive Bayes model:

$$P(y|x) = \frac{P(x|y) \cdot P(y)}{P(x)}$$

Where:

- $P(y|x)$: Posterior probability (probability of class y given data x)
- $P(x|y)$: Likelihood (probability of data x given class y)
- $P(y)$: Prior probability of class y
- $P(x)$: Evidence (normalizing constant, independent of y)

Naive Bayes Assumption

The "naive" assumption is that all features are **conditionally independent** given the class label:

$$P(x|y) = \prod_{i=1}^{n} P(x_i|y)$$

Where $x = \{x_1, x_2, \ldots, x_n\}$ represents the feature vector.

Model Prediction

The model predicts the class y that maximizes the posterior probability $P(y|x)$:

$$y = \arg\max_y P(y|x) = \arg\max_y P(y) \cdot \prod_{i=1}^{n} P(x_i|y)$$

Advantages

1. Simple and computationally efficient.

2. Works well for high-dimensional data.
3. Requires a small amount of training data.
4. Performs well in text classification, spam filtering, and sentiment analysis.

Limitations

1. The assumption of feature independence may not hold in real-world data.
2. Struggles with small datasets where conditional probabilities are difficult to estimate.
3. Sensitive to imbalanced data unless proper priors are used.

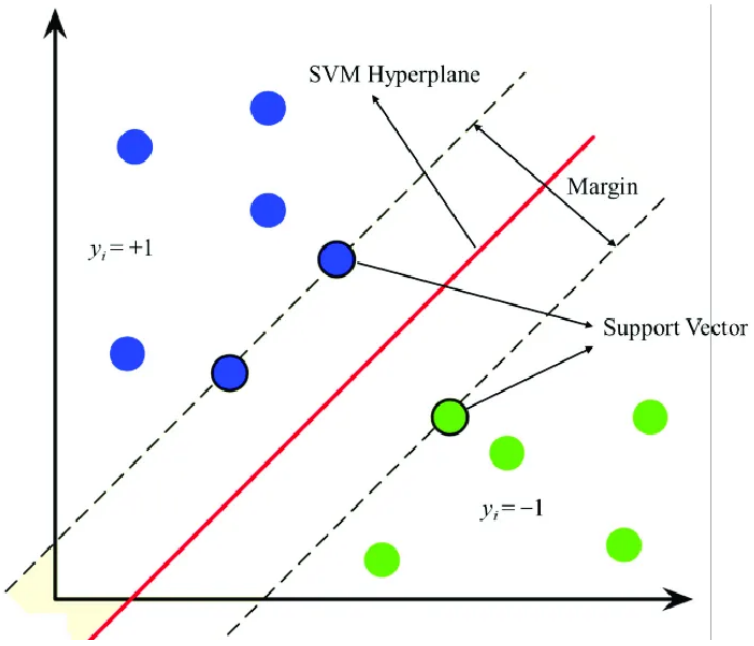
Applications

1. **Text Classification**: Spam email detection, sentiment analysis.
2. **Document Categorization**: Sorting news articles into topics.
3. **Medical Diagnosis**: Predicting diseases based on symptoms.
4. **Recommender Systems**: Classifying user preferences.

Support Vector Machine (SVM):

- **Support Vector Machine (SVM)** is a supervised learning algorithm primarily used for classification tasks, though it can also be used for regression.
- It finds the optimal decision boundary (or hyperplane) that separates classes in the feature space with the **maximum margin**.

Key Concepts



1. Hyperplane

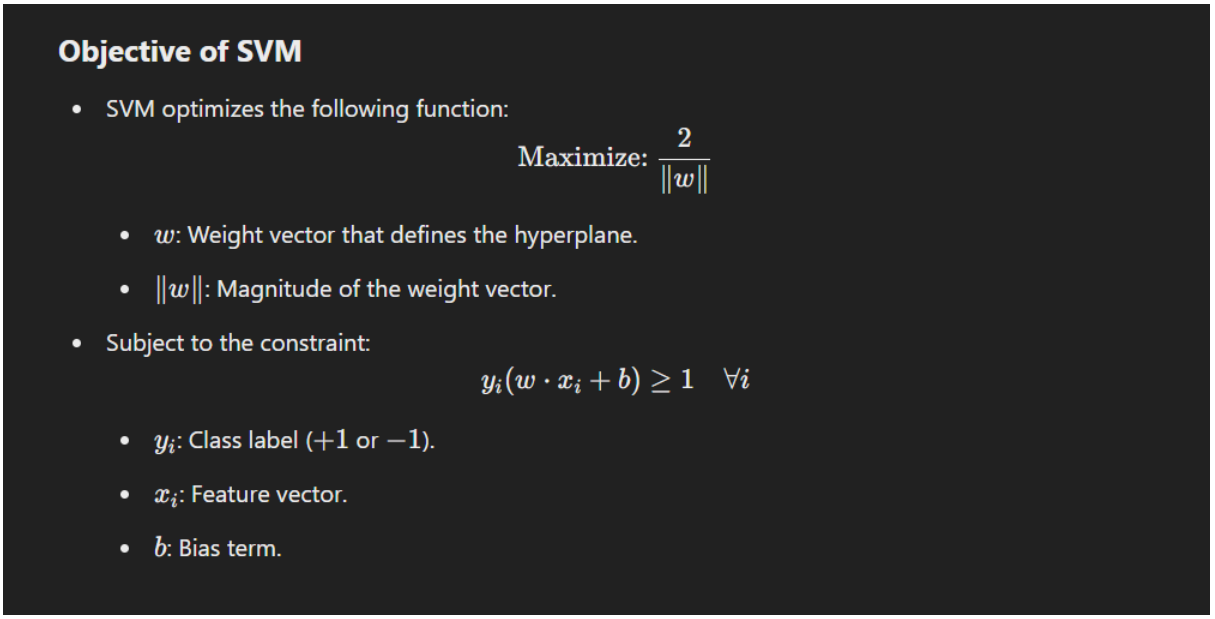
- A hyperplane is a decision boundary that separates data points into different classes.
- In an n- dimensional feature space:
  - A hyperplane is an (n−1) dimensional subspace (e.g., a line in 2D or a plane in 3D).

2. Margin

- The margin is the distance between the hyperplane and the closest data points from each class.
- SVM aims to maximize this margin to ensure the classifier is robust and generalizes well to unseen data.

3. Support Vectors

- Support vectors are the data points that lie closest to the hyperplane.
- These points are critical as they determine the position and orientation of the hyperplane.



Steps for Implementing SVM

1. **Prepare Data**: Preprocess and scale the features.
2. **Choose Kernel**: Select an appropriate kernel function (e.g., linear, RBF).
3. **Train the Model**: Use an SVM library to train the model.
4. **Hyperparameter Tuning**: Adjust , kernel parameters (,) for optimal performance.
5. **Evaluate Performance**: Use metrics like accuracy, precision, recall, or F1-score.

Kernel Functions

1. **Linear Kernel**
2. **Polynomial Kernel**
3. **Radial Basis Function (RBF)**
4. **Sigmoid Kernel**

Advantages

1. Effective for high-dimensional data.
2. Works well with a clear margin of separation.

Disadvantages

1. Computationally expensive for large datasets.
2. Choice of kernel and hyperparameters (e.g., ,) can be challenging.
3. Sensitive to noise and outliers.

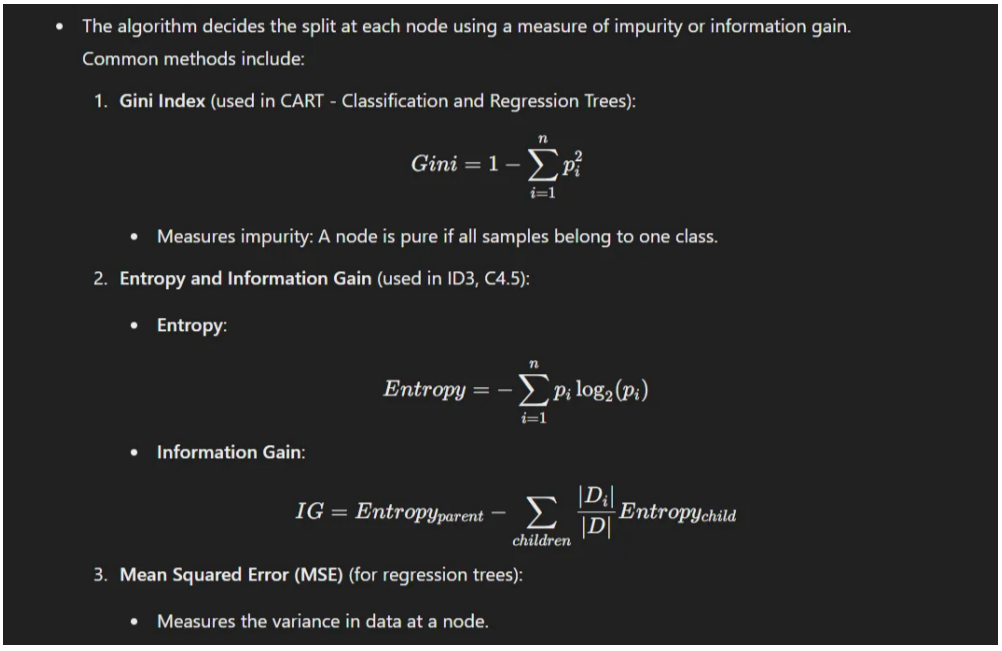
Applications

1. Text classification (e.g., spam detection).
2. Image recognition.
3. Bioinformatics (e.g., protein classification).
4. Financial analysis.

Decision Trees:

- A **decision tree** is a supervised learning algorithm used for both **classification** and **regression** tasks.
- It works by recursively splitting the data into subsets based on feature values to build a tree-like structure.
- The tree consists of:
  - **Root Node**: The top node representing the entire dataset.
  - **Internal Nodes**: Decision points based on feature values.
  - **Leaf Nodes**: Terminal nodes that provide the output (class label or predicted value).

Splitting Criteria



Advantages

1. **Simple to understand** and interpret (visual representation).
2. Requires little data preprocessing (no need for normalization or scaling).
3. Handles both numerical and categorical data.
4. Can model non-linear relationships.

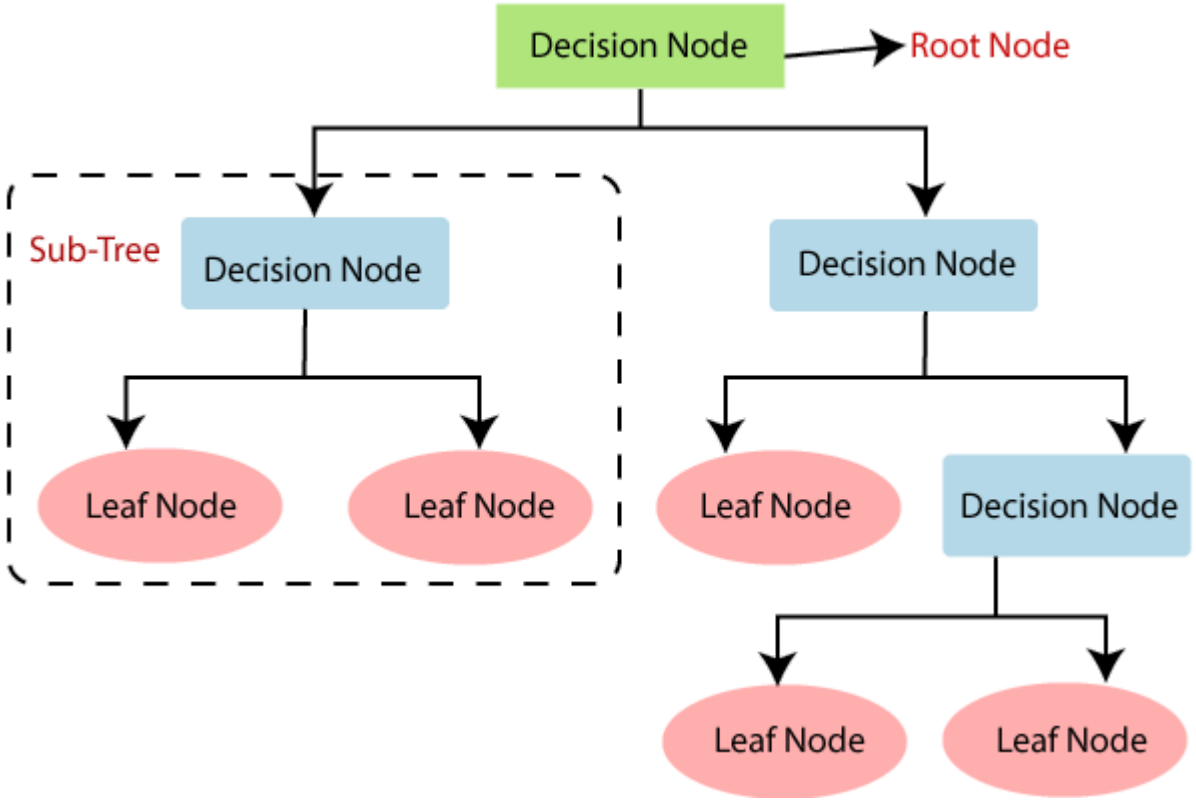
Disadvantages

1. Prone to **overfitting** if the tree grows too large.
2. Small changes in data can lead to a different tree structure (unstable).
3. Bias towards features with more categories (e.g., ID3 prefers features with many unique values).

Steps to Build a Decision Tree

1. **Choose the Root Node**: Select the feature that provides the best split (highest information gain or lowest impurity).
2. **Split Data**: Divide the dataset based on the selected feature.
3. **Repeat Recursively**: Apply the same process for child nodes until stopping criteria are met.

4. **Assign Class/Value**: Assign a class label (for classification) or a predicted value (for regression) at the leaf nodes.



Types of Decision Trees

1. Classification Tree

- Used when the target variable is categorical.
- Outputs class labels.

2. Regression Tree

- Used when the target variable is continuous.
- Outputs numerical values.

Metrics for Evaluation

- **For Classification Trees**:
  - Accuracy
  - Precision, Recall, and F1-score

- Confusion Matrix

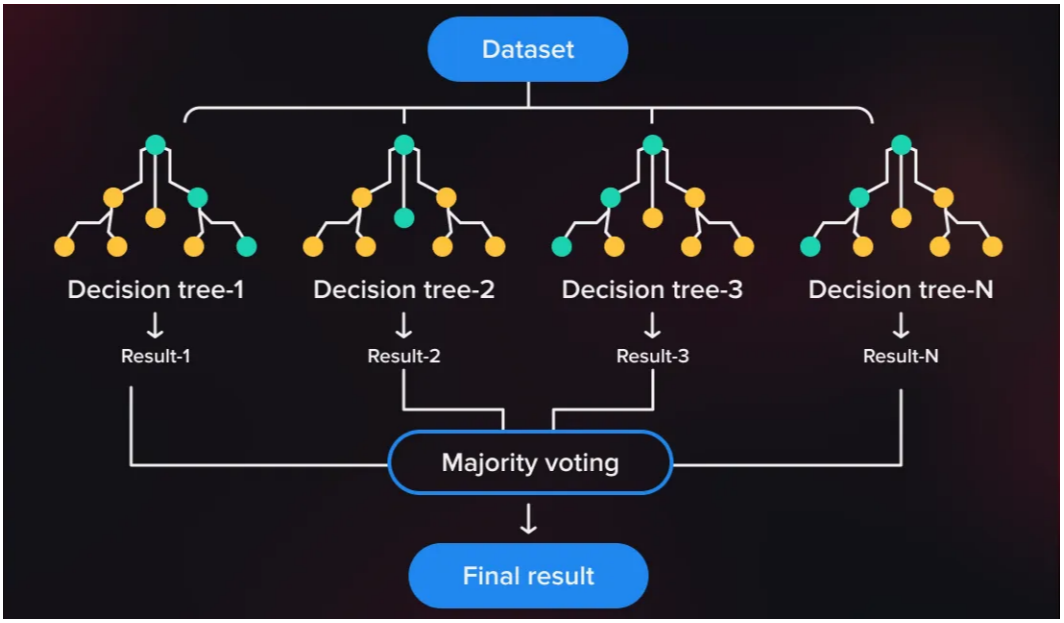
- **For Regression Trees**:
  - Mean Absolute Error (MAE)
  - Mean Squared Error (MSE)
  - R^2score

Applications

1. Medical diagnosis (e.g., disease classification).
2. Customer segmentation and churn prediction.
3. Credit risk analysis.
4. Recommendation systems.

Random Forest:

- **Random Forest** is a supervised learning algorithm that combines multiple decision trees to make robust predictions.
- It is an **ensemble method**, specifically a type of **bagging** technique, where each tree is trained on a random subset of the data and features.



Key Features

1. **Works for Classification and Regression**:
    - Classification: Predicts a class label.
    - Regression: Predicts a continuous value.
2. **Reduces Overfitting**:
    - By averaging the results of multiple trees, Random Forest reduces the risk of overfitting seen in individual decision trees.
3. **Handles Missing Data and Outliers**:
    - It is robust to noisy data and outliers.

How It Works

1. Ensemble Learning

- Combines predictions from multiple models to improve accuracy and stability.
- Random Forest is based on **bagging** (Bootstrap Aggregating), where:
    - Each tree is trained on a bootstrap sample (random subset) of the dataset.
    - Predictions from all trees are aggregated (e.g., majority voting for classification, averaging for regression).

2. Random Feature Selection

- During tree construction:
    - At each split, a random subset of features is considered, rather than all features.
    - This introduces diversity among the trees and reduces correlation between them.

3. Aggregation

- The final prediction is based on:
    - **Classification**: Majority vote from all trees.
    - **Regression**: Average prediction from all trees.

Steps in Random Forest

1. **Bootstrap Sampling**:
   - Create multiple subsets of the dataset by sampling with replacement.
2. **Train Decision Trees**:
   - Train a decision tree on each subset.
   - At each split, use a random subset of features to determine the best split.
3. **Aggregate Predictions**:
   - Combine predictions from all trees for the final result.

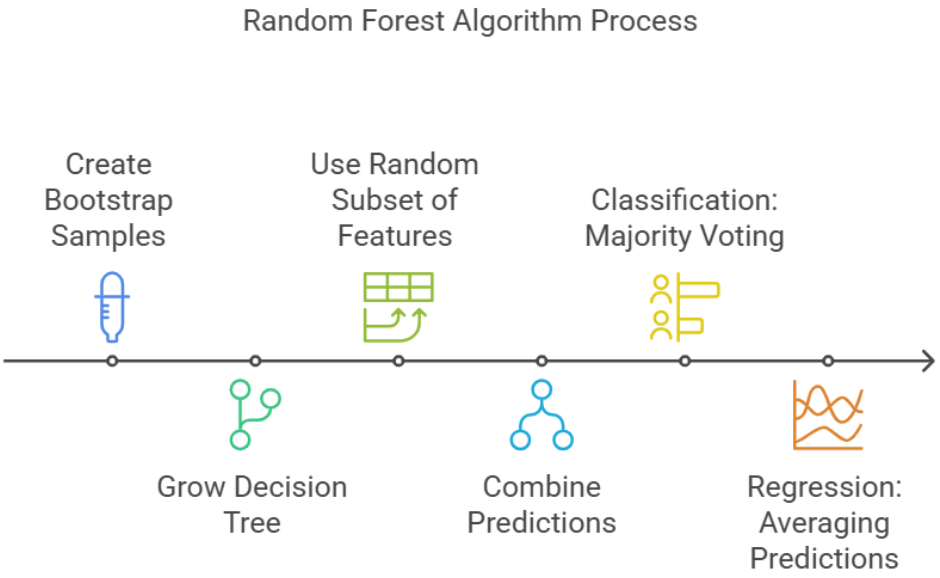
Evaluation Metrics

- **Classification**:
  - Accuracy, Precision, Recall, F1-score.
- **Regression**:
  - Mean Absolute Error (MAE), Mean Squared Error (MSE), Score.

    R^2

Algorithm Workflow

1. Randomly sample the dataset to create bootstrap samples.
2. Grow a decision tree for each bootstrap sample:
   - At each split, use a random subset of features.
3. Combine predictions from all trees:
   - Classification: Majority voting.
   - Regression: Averaging predictions.



Advantages Over Decision Trees

- Decision Trees:
  - Tend to overfit on the training data.
  - Sensitive to noise and outliers.
- Random Forest:
  - Reduces overfitting by averaging multiple trees.
  - Provides better generalization.

Applications

1. **Classification Tasks**:
   - Spam detection, fraud detection, medical diagnosis.
2. **Regression Tasks**:
   - House price prediction, stock market analysis.
3. **Feature Selection**:
   - Random Forest ranks features by their importance, helping in feature engineering.
4. **Anomaly Detection**:
   - Detecting outliers or rare events.